

OAuth 2.0: Security Considerations

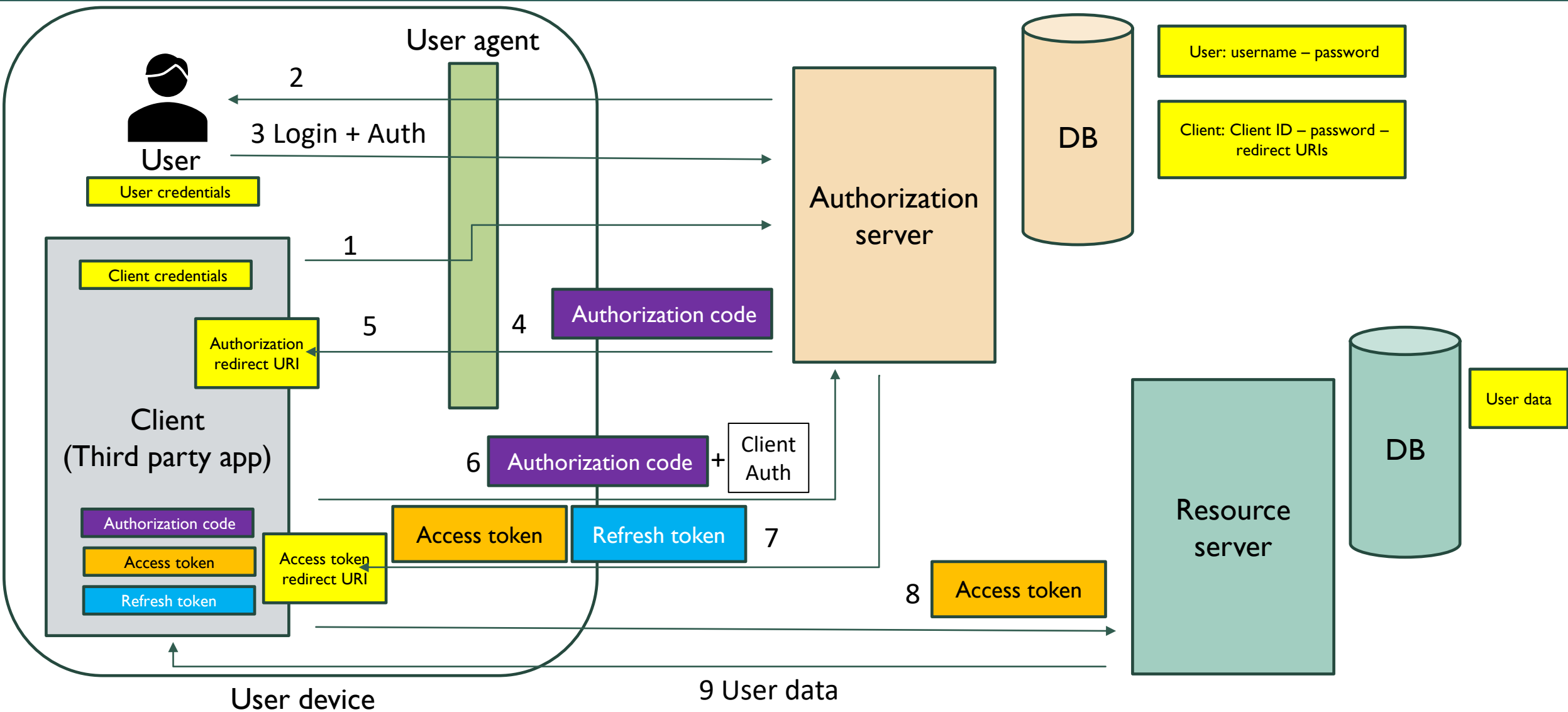
Simone Bussa
Politecnico di Torino

OAuth 2.0: Introduction

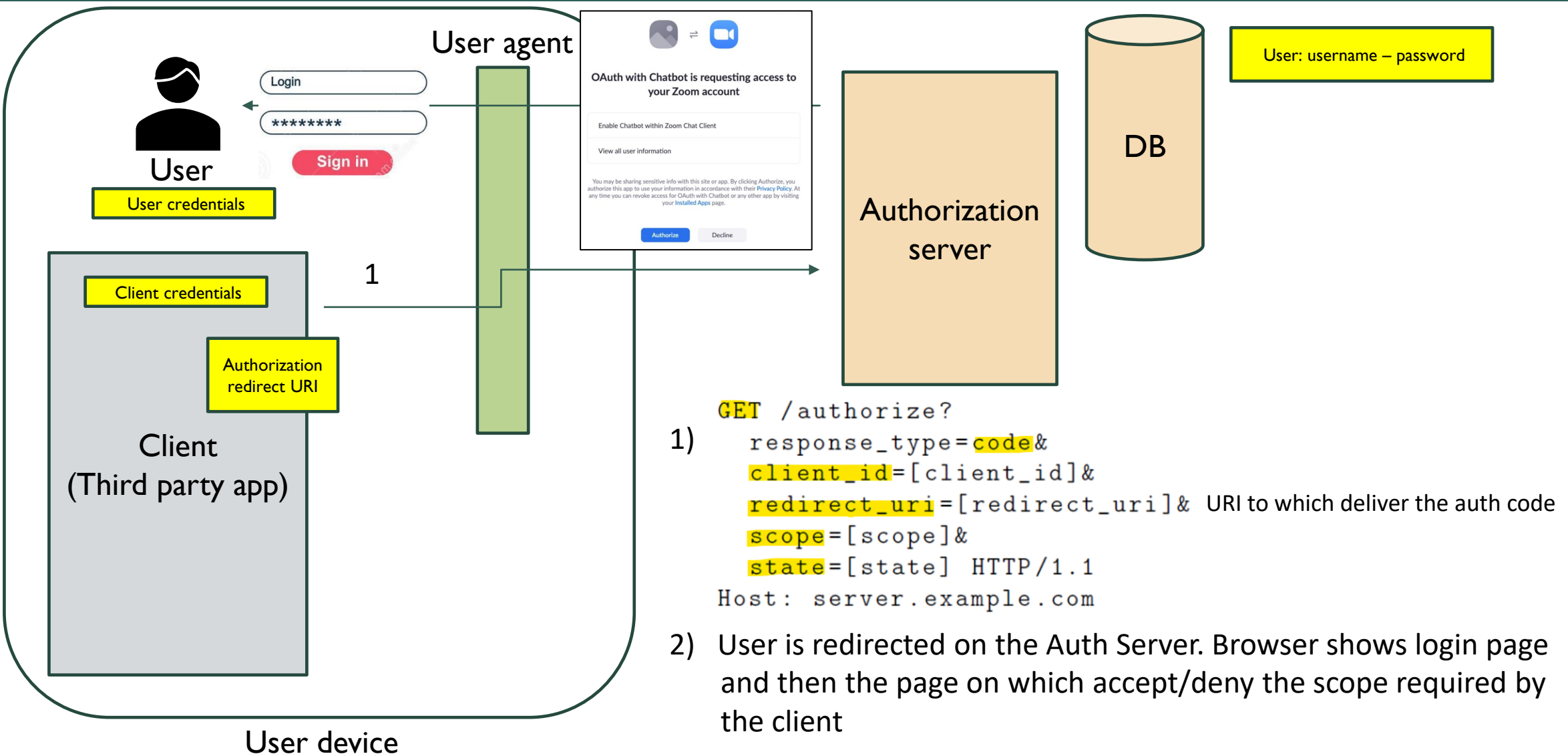


- Authorization framework
- Temporary access to a resource in a well-defined scope to third-party app
- Does not deal with how the resource owner authenticates

OAuth 2.0: Authorization code flow

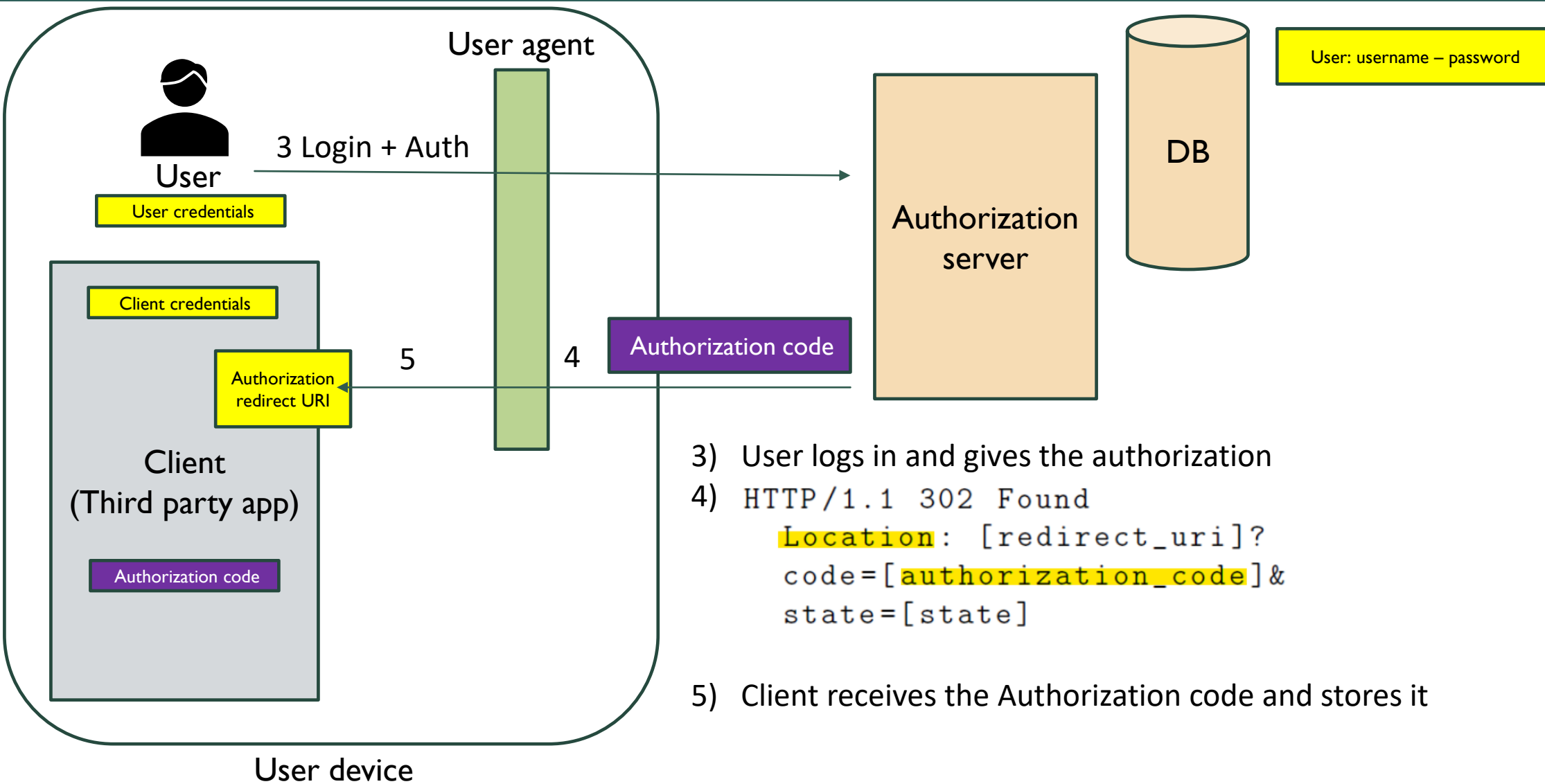


OAuth 2.0: Authorization code flow





OAuth 2.0: Authorization code flow



3) User logs in and gives the authorization

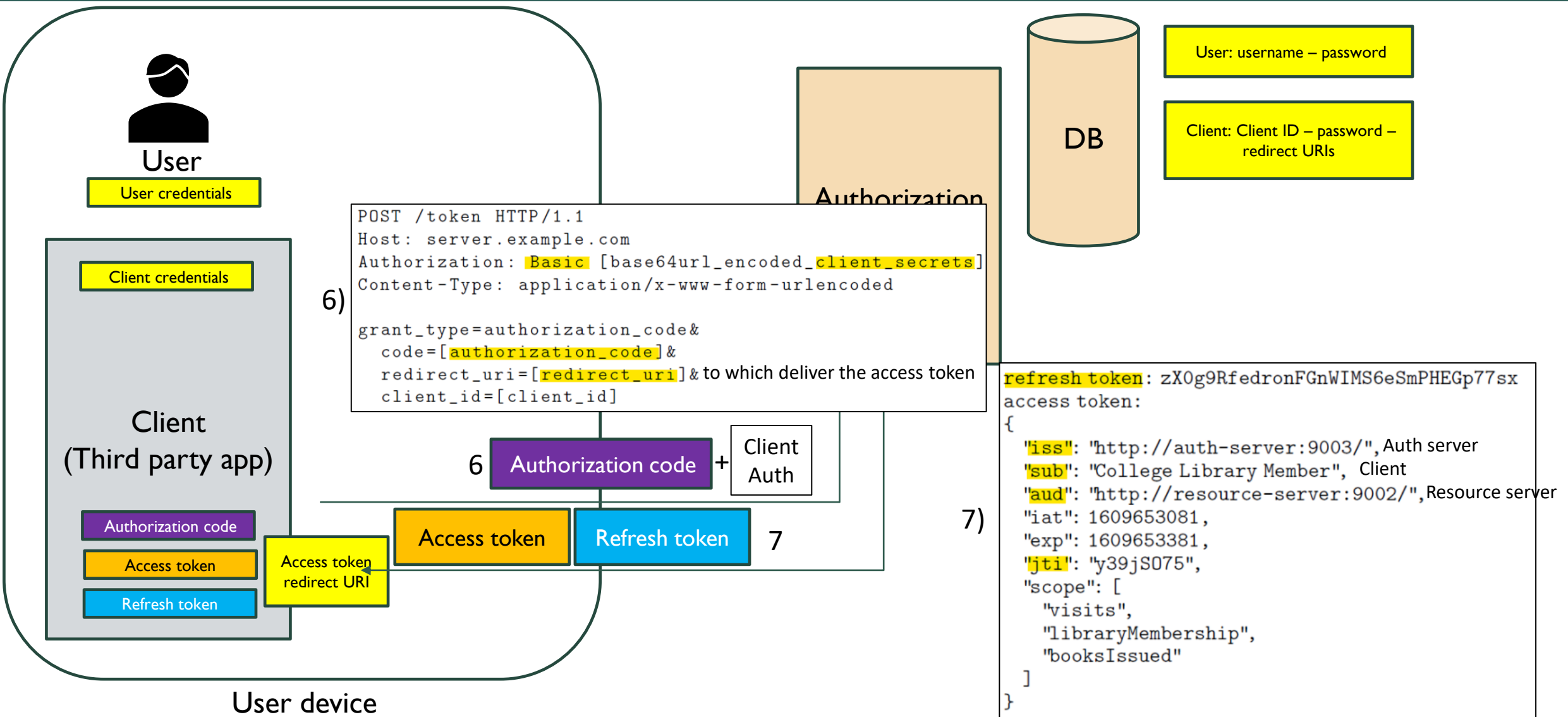
4) HTTP/1.1 302 Found

```
Location: [redirect_uri]?  
code=[authorization_code]&  
state=[state]
```

5) Client receives the Authorization code and stores it

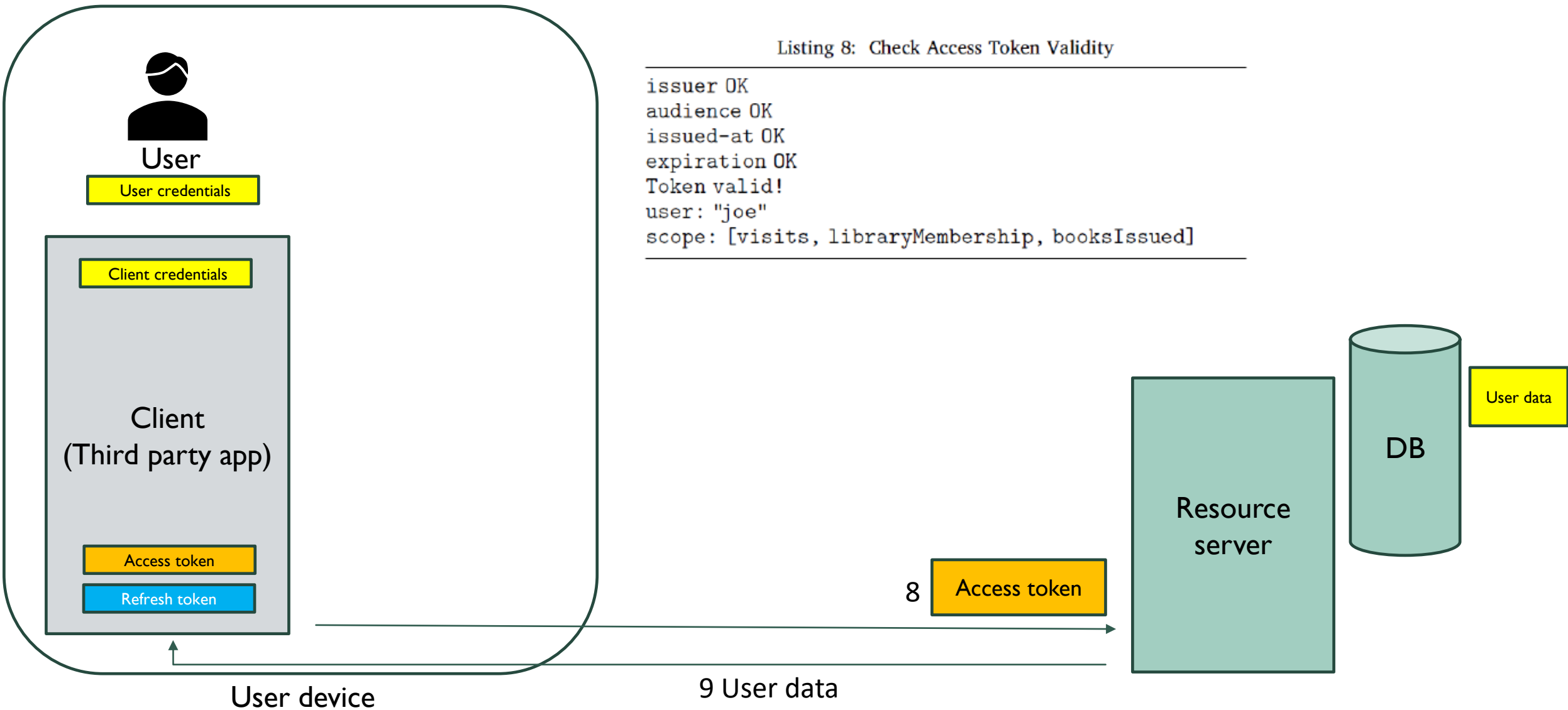


OAuth 2.0: Authorization code flow

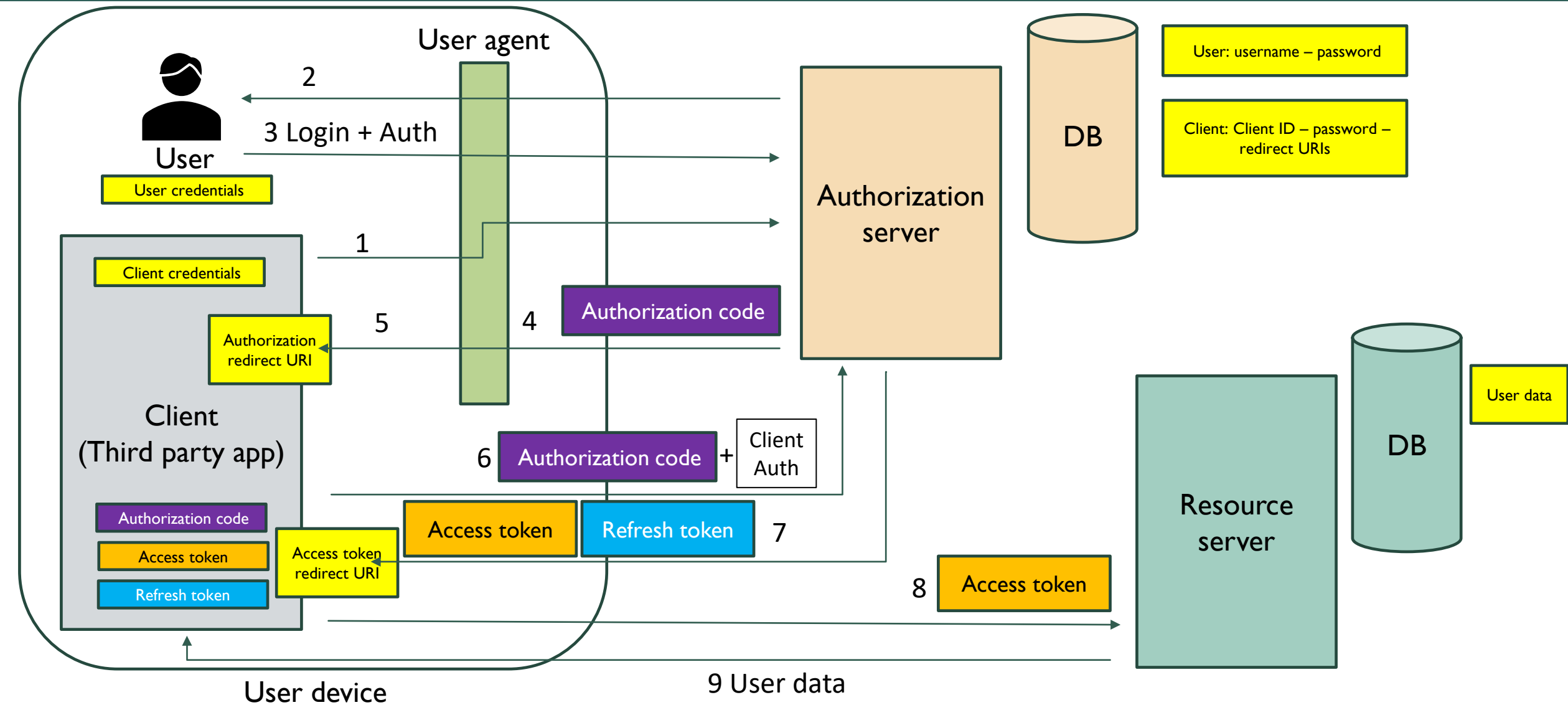




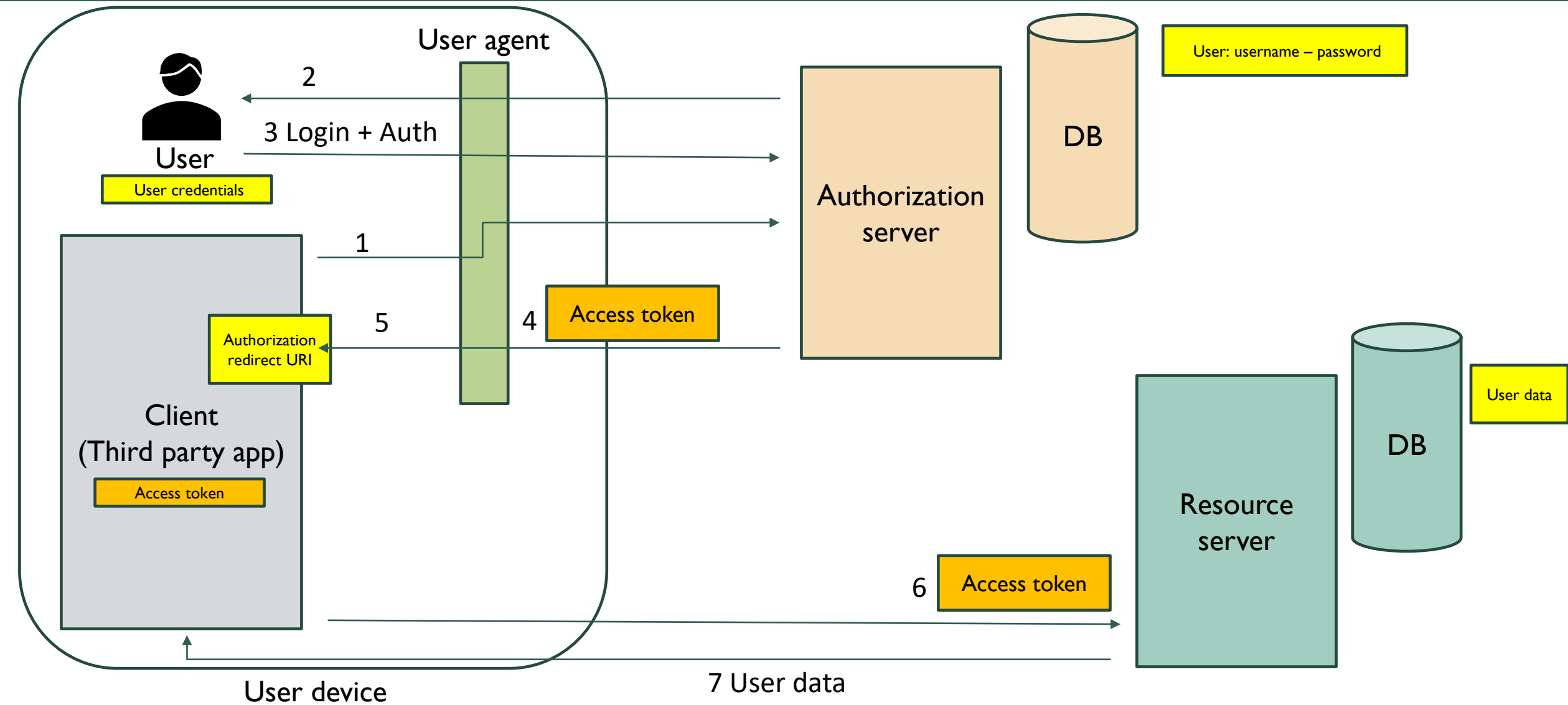
OAuth 2.0: Authorization code flow



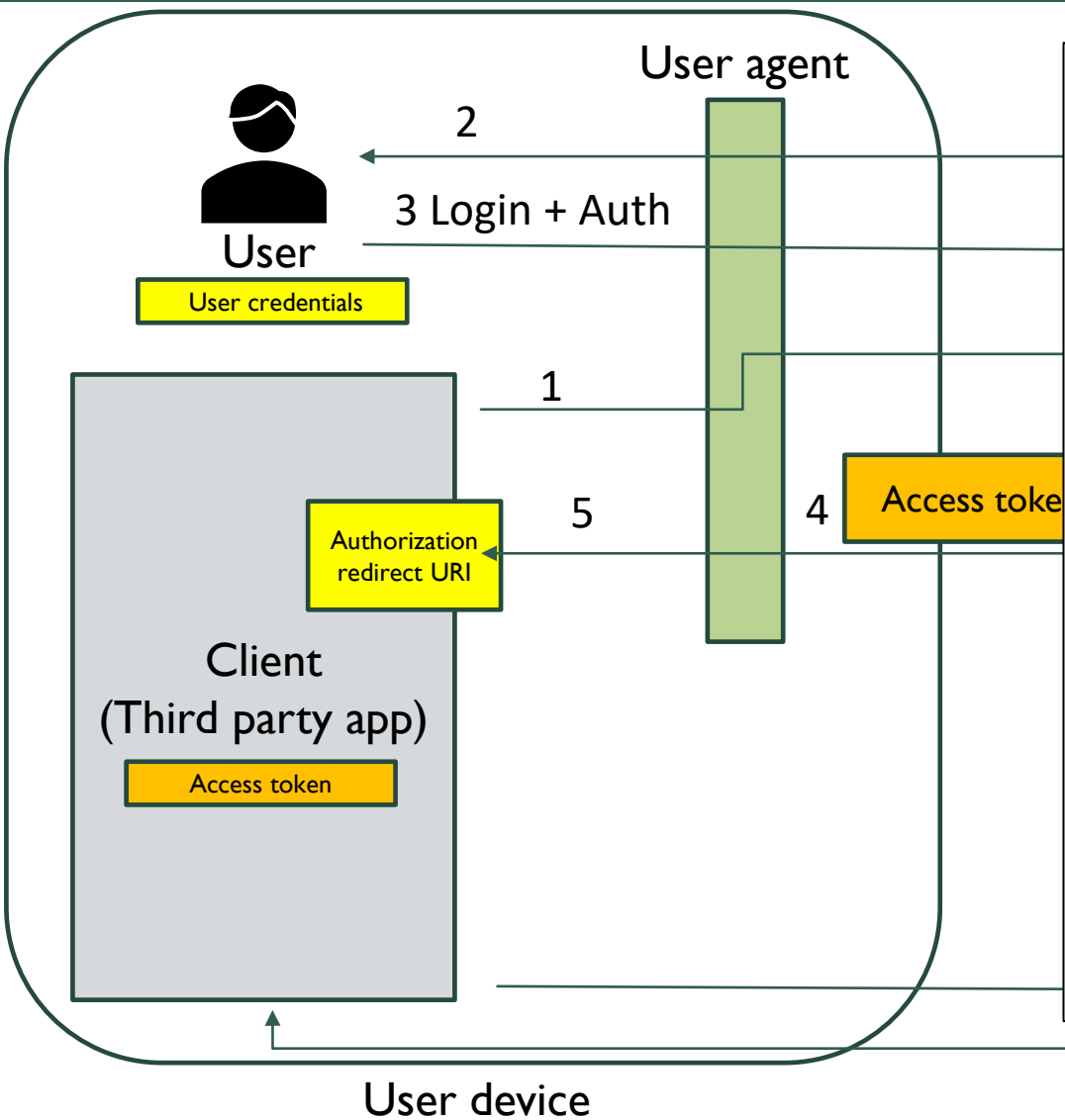
OAuth 2.0: Authorization code flow



OAuth 2.0: Implicit grant flow



OAuth 2.0: Implicit grant flow

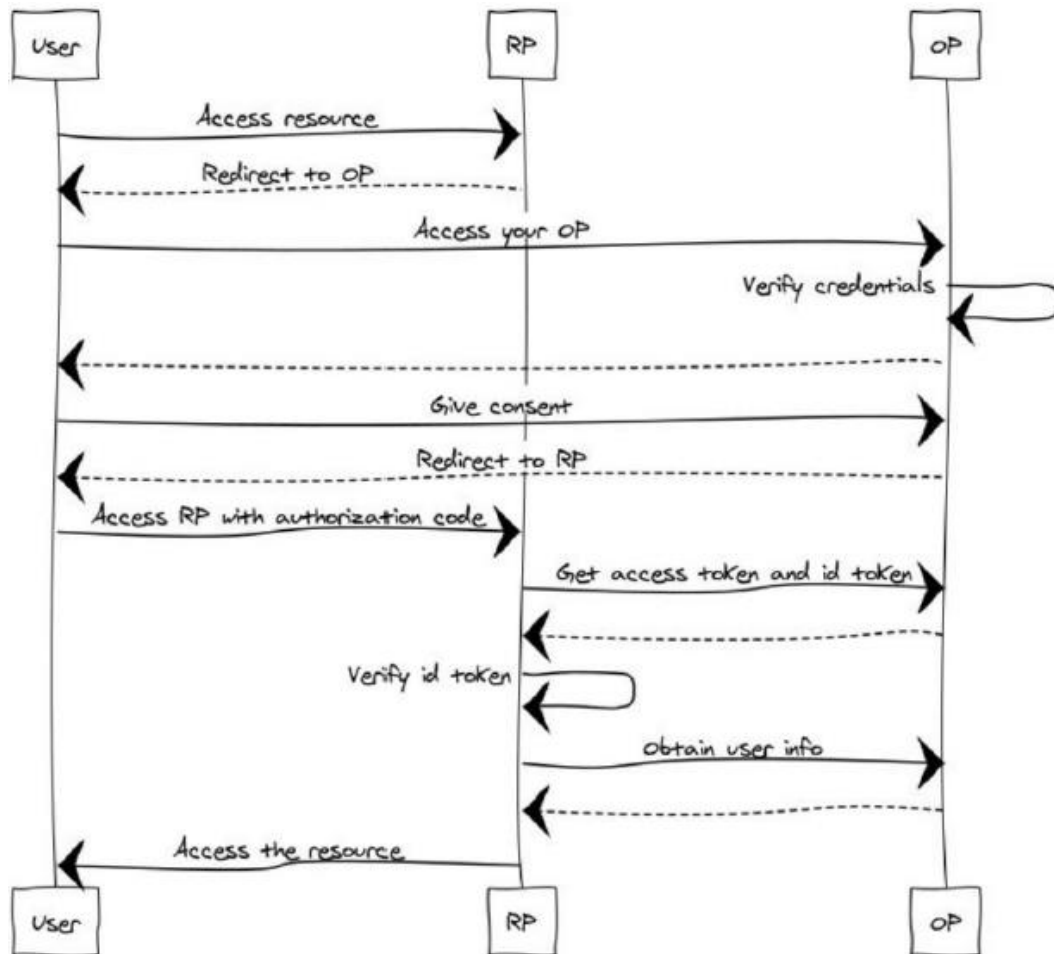


- There is no Client authentication (perfect for Public clients)
- Allows to obtain the access token quickly (without the need of a second interaction with the Auth server)
- Does not provide the emission of the Refresh token
- Access token included in the URI fragment

```
HTTP/1.1 302 Found
Location: http://example.com/cb#access_token=2YotnFZFEjr1zCsicMWpAA
&state=xyz&token_type=example&expires_in=3600
```

- A sniffer cannot read the fragment (fragment is retained locally inside the browser)
- A request to the web hosted client resource is done (<http://example.com/cb>) without passing the fragment
- A web page is returned with an embedded script capable of accessing the full redirect uri and extracting the fragment
- The browser loads the page, executes the script, extracts the access token and return it to the client

OAuth 2.0: OpenID Connect



- There is an additional endpoint, UserInfo endpoint (UIE), that the Client can contact to get personal attribute of the authenticated user
- Client exchanges the Authentication code for Access token + ID token + Refresh token

```
{
  "iss": "http://server.example.com",
  "sub": "248289761001",
  "aud": "s6BhdRkqt3",
  "nonce": "n-0S6_WzA2Mj",
  "exp": 1311281970,
  "iat": 1311280970,
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe",
  "gender": "female",
  "birthdate": "0000-10-31",
  "email": "janedoe@example.com",
  "picture": "http://example.com/janedoe/me.jpg"
}
```

The Client exchanges ID token for user attributes

OAuth 2.0: Client registration

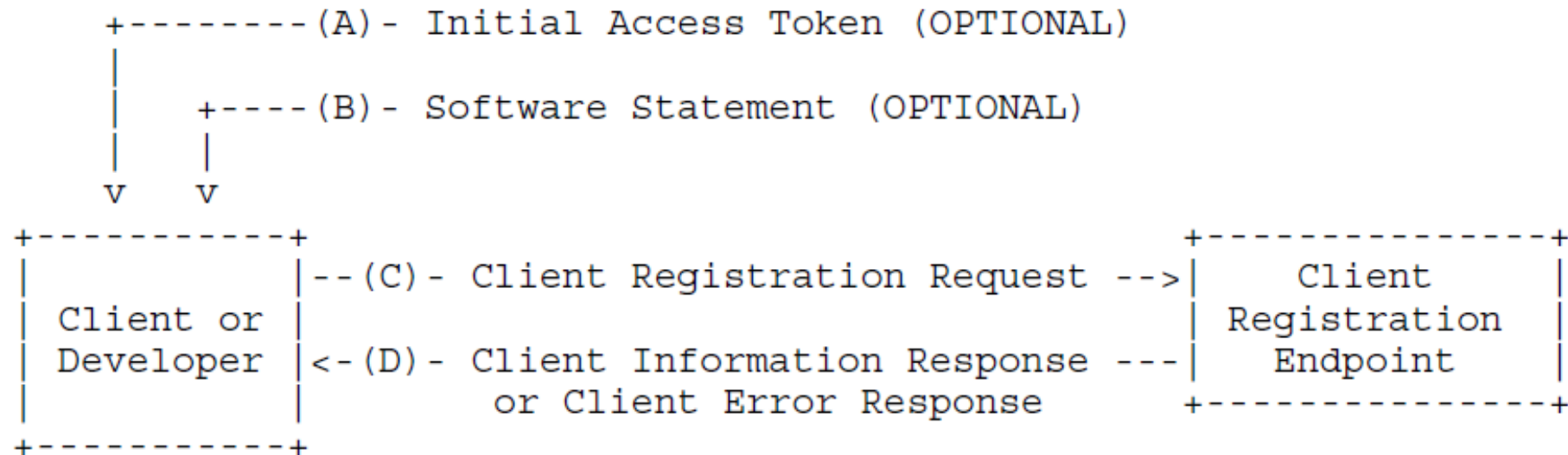


- **Two types** of clients:
 - **Confidential**: able to maintain confidential client_secrets. Example: **web applications** (that reside on a server)
 - **Public**: not able to maintain client_secrets. Example: **browser-based applications** (client code downloaded by the browser and easily accessible to the user), **native application** (source code or binary accessible to the user)

OAuth 2.0: Client registration



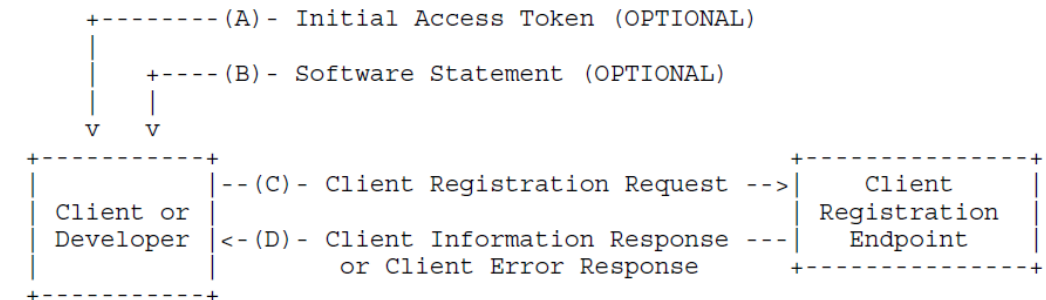
- Dynamic client registration protocol (**RFC 7591**)
 - Client obtains a **client ID** to use in all OAuth requests, Server obtains Client **metadata**



OAuth 2.0: Client registration



- Initial Access token (optional)
 - Token optionally issued by an authorization server to a client (or client developer), before the Client registration process, to be used in order to access Client Registration endpoint



- Used to limit/filter the clients that can dynamically register to that Authorization server (DoS)
- Software statement (optional)
 - Client metadata can be sent as normal self-asserted JSON objects or through a Software Statement
 - In this last case, there is an authority that validates Client metadata and inserts them into a Signed JWT (Software Statement). There is a federation between the Issuer authority and the Auth server
 - Self-asserted JSON data and Software Statement can coexist (in this case some fields are validated and some are self-asserted)

OAuth 2.0: Client registration

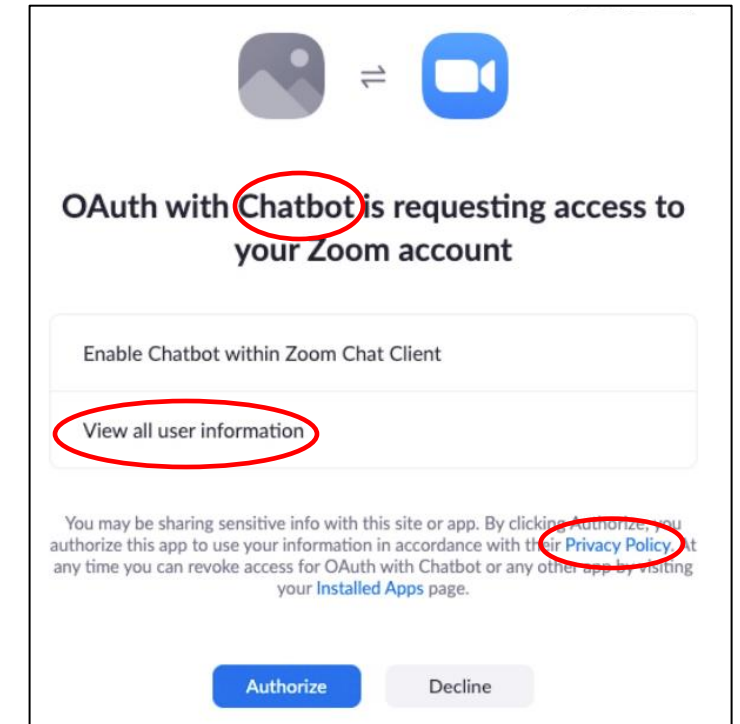
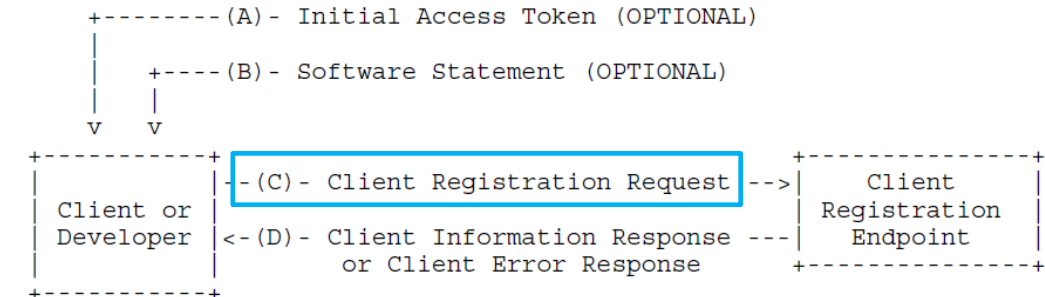


■ Client registration request

- **Access token** (optional) + **client metadata** (JSON self-asserted data or/and Software Statement)

■ Metadata fields:

- Redirect_uris
- Token_endpoint_auth_method (method that the Client can use to authenticate with the Auth server to get Access token, see more later)
- Grant_types (authorization_code, implicit, password, refresh_token etc)
- Client_name (to be presented to the user during authorization)
- Client_uri (a web page that provides info about the client)
- Scope (useful for public clients that cannot authenticate themselves)
- Policy_uri (web page that describes how user data are managed, for privacy issues)
- Jwks or jwks_uri (JSON Web Key Set, public keys of the client)



OAuth 2.0: Client registration

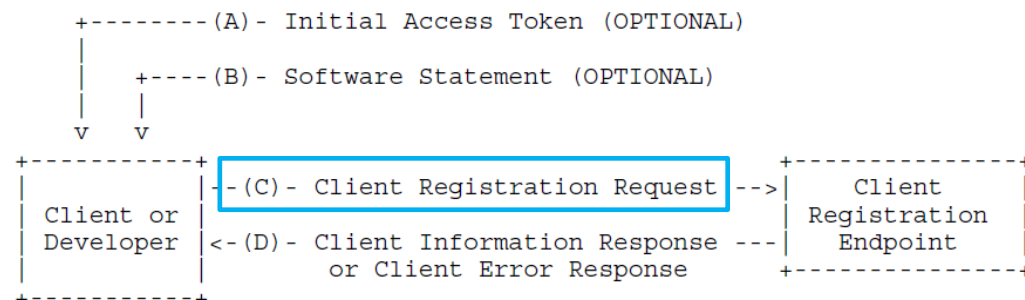


- Examples of Client registration requests

- Access token + self-asserted metadata

```
POST /register HTTP/1.1
Content-Type: application/json
Accept: application/json
Authorization: Bearer ey23f2.adfj230.af32-developer321
Host: server.example.com
```

```
{
  "redirect_uris": ["https://client.example.org/callback",
    "https://client.example.org/callback2"],
  "client_name": "My Example Client",
  "client_name#ja-Jpan-JP":
    "\u30AF\u30E9\u30A4\u30A2\u30F3\u30C8\u540D",
  "token_endpoint_auth_method": "client_secret_basic",
  "policy_uri": "https://client.example.org/policy.html",
  "jwks": {"keys": [{
    "e": "AQAB",
    "n": "nj3YJwsLUF19BmpAbkOswCNVx17Eh9wMO-_AReZwBqfaWFcfG
HrZXsIV2VMCNVNU8Tpb4obUaSXcRcQ-VMsfQPJm9IzgtRdAY8NN8Xb7PEcYyk
1BjvTtuPbpzIaqyiUepzUXNDFuAOOkriOl3Wmf1PUUgMKULBN0EUd1fpOD70p
RM0rlp_gg_WNUKoW1V-3keYUJoXH9NztEDm_D2MQXj9eGOJJ8yPgGL8PAZMLLe
2R7jb9TxOCPEDE7tY_TU4nFP1xptw59A42mldEmViXsKQt60s1SLboazxFKve
qXC_jpLUt22OC6GUG63p-REw-ZOr3r845z50wMuzifQrMI9bQ",
    "kty": "RSA"
  ]}],
  "example_extension_parameter": "example_value"
}
```



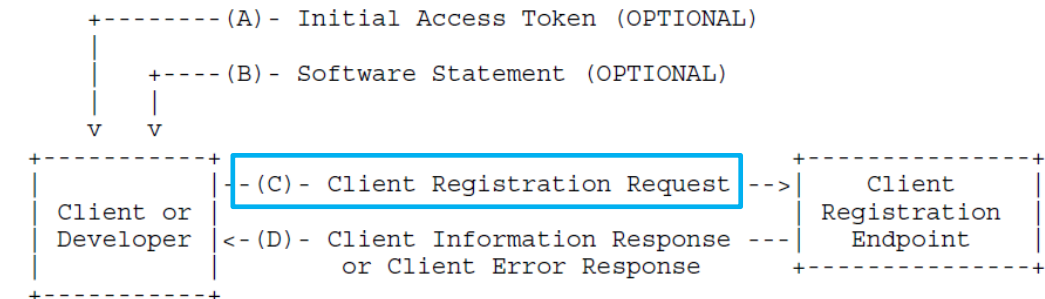
OAuth 2.0: Client registration



- Examples of Client registration requests
 - NO Access token + self-asserted metadata + software statement metadata

```
POST /register HTTP/1.1
Content-Type: application/json
Accept: application/json
Host: server.example.com
```

```
{
  "redirect_uris": [
    "https://client.example.org/callback",
    "https://client.example.org/callback2"
  ],
  "software_statement": "eyJhbGciOiJSUzI1NiJ9.eyJzb2d2Z0d2FyZV9pZCI6IjROUkIxLTBYWkFCWkk5RTYtNVNNM1IiLCJjbG1bnRfbmFtZSI6IktV4YW1wbGUgU3RhdGVtZW50LWJhc2VkiENsaWVudCIsImNsYWVudF91cmkiOiJodHRwc2ovL2NsaWVudC5leGFtcGxlLm5ldC8ifQ.GHfL4QNIrQwL18BSRdE595T9jzbzqa06R9BT8w409x9oIcKaZo_mt15riEXHa zdISUvDIzhtiyNrSHQ8K4TvqWxH6uJgcmoodZdPwmWRIEYbQDLqPNxREtYn0 5X3AR7ia4FRjQ2ojZjk5fJqJdQ-JcfxyhK-P8BAWBd6I2LLA77IG32xtbhxY fHX7VhuU5ProJO8uvu3Ayv4XRhLZJY4yKfmyjiiKiPNe-Ia4SMY_d_QSWxsk U5XIQl5Sa2YRPMbDRXttm2TfnZM1xx70DoYi8g6czz-CPGRi4SW_S2RKHIJf IjoI3zTJ0Y2oe0_EJAiXbL6OyF9S5tKxDXV8JIndSA",
  "scope": "read write",
  "example_extension_parameter": "example_value"
}
```



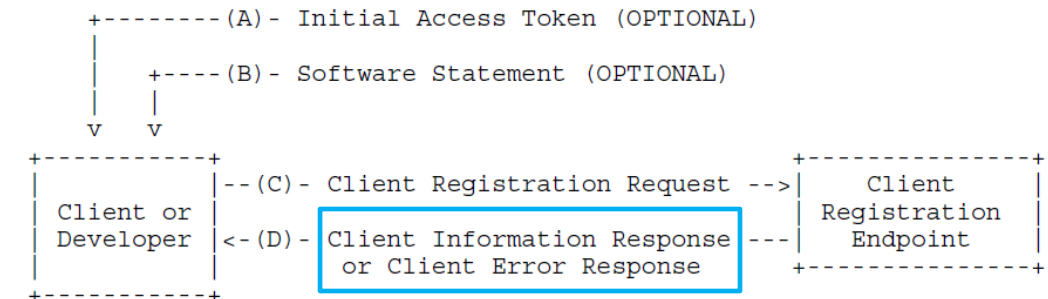
OAuth 2.0: Client registration



- Client registration response
 - **Registered metadata** (may differ from the ones in the requests, the client should accept the modification or try with a new registration) + client ID + client secret (OPTIONAL)

```
HTTP/1.1 201 Created
Content-Type: application/json
Cache-Control: no-store
Pragma: no-cache
```

```
{
  "client_id": "s6BhdRkqt3",
  "client_secret": "cf136dc3c1fc93f31185e5885805d",
  "client_id_issued_at": 2893256800,
  "client_secret_expires_at": 2893276800,
  "redirect_uris": [
    "https://client.example.org/callback",
    "https://client.example.org/callback2"],
  "grant_types": ["authorization_code", "refresh_token"],
  "client_name": "My Example Client",
  "client_name#ja-Jpan-JP":
    "\u30AF\u30E9\u30A4\u30A2\u30F3\u30C8\u540D",
  "token_endpoint_auth_method": "client_secret_basic",
  "logo_uri": "https://client.example.org/logo.png",
  "jwks_uri": "https://client.example.org/my_public_keys.jwks",
  "example_extension_parameter": "example_value"
}
```

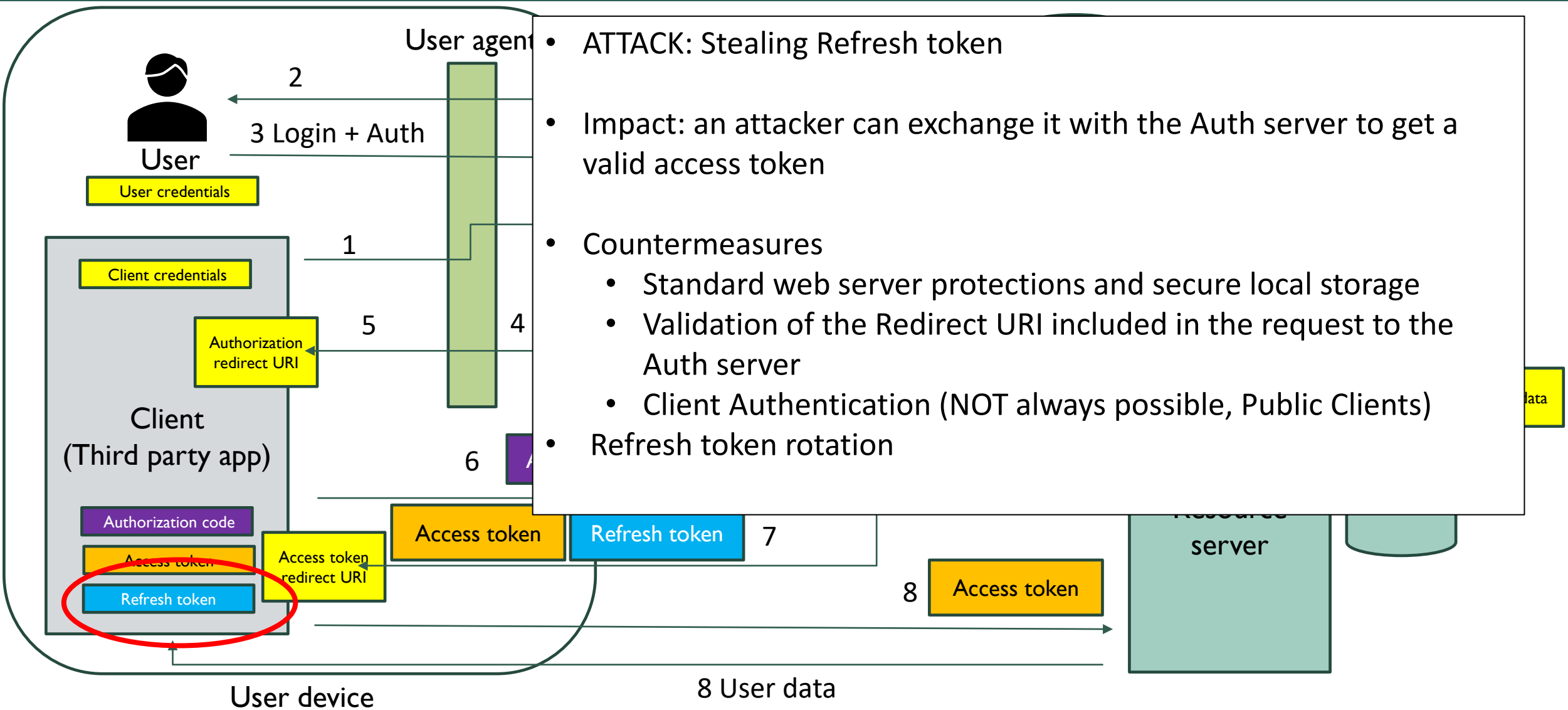


OAuth 2.0: Client registration



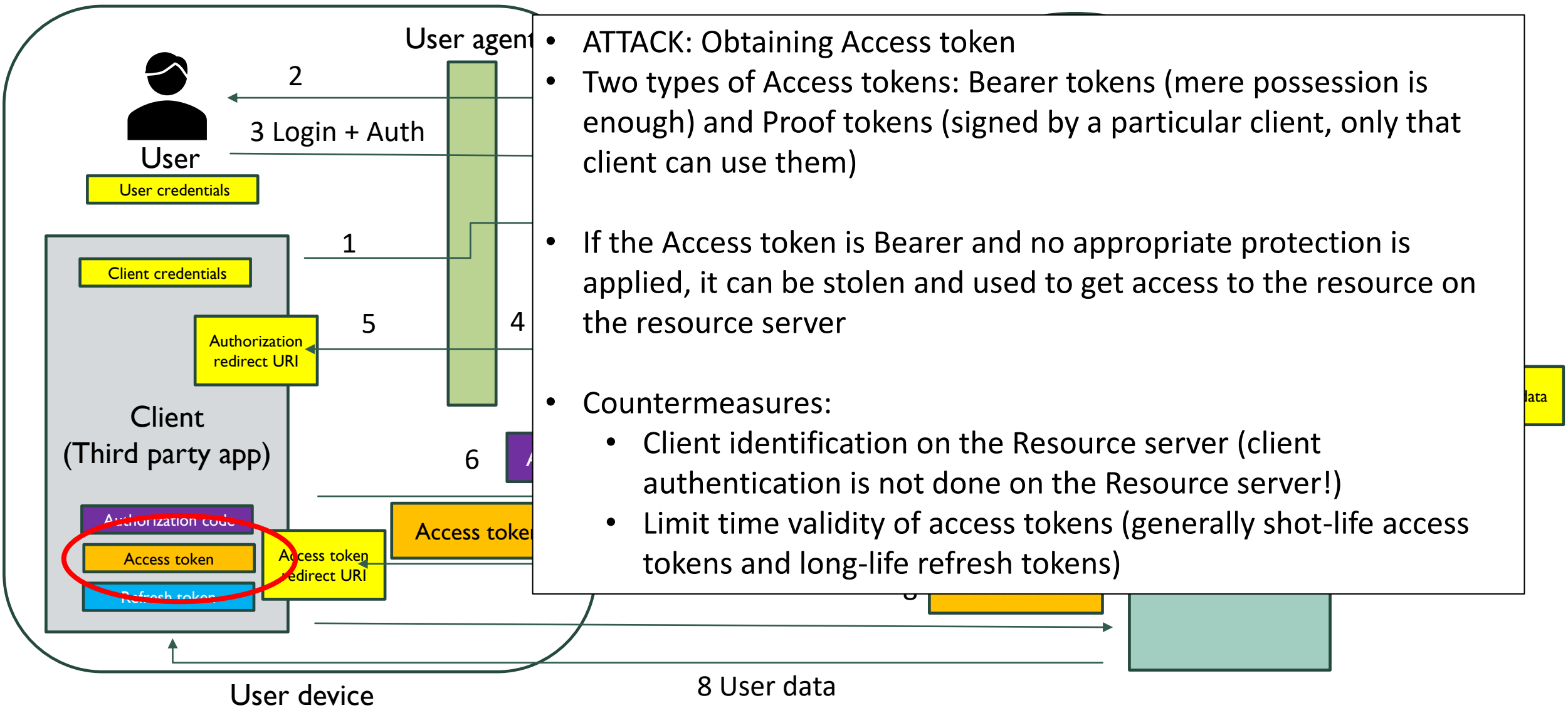
- Security considerations:
 - Both Registration requests and responses MUST be protected using TLS (protect metadata from modifications, protect client credentials)
 - Few privacy concerns since the protocol deals with software information and NOT people
 - Validation of the uris displayed to the user during authorization(client_uri, policy_uri etc)
 - Public clients typically register with token_endpoint_auth_method = “none” and use Implicit grant type
 - Since they cannot provide a secure long-term storage, in some cases it is possible they register with the Auth server every time the Client is loaded (short-lived clients)
 - Proliferation of dead client identifiers and metadata on the Auth server (memory consumption)
 - Expiration time or info maintained by clients and pushed on the server during OAuth protocol

OAuth 2.0: Attacks against Client

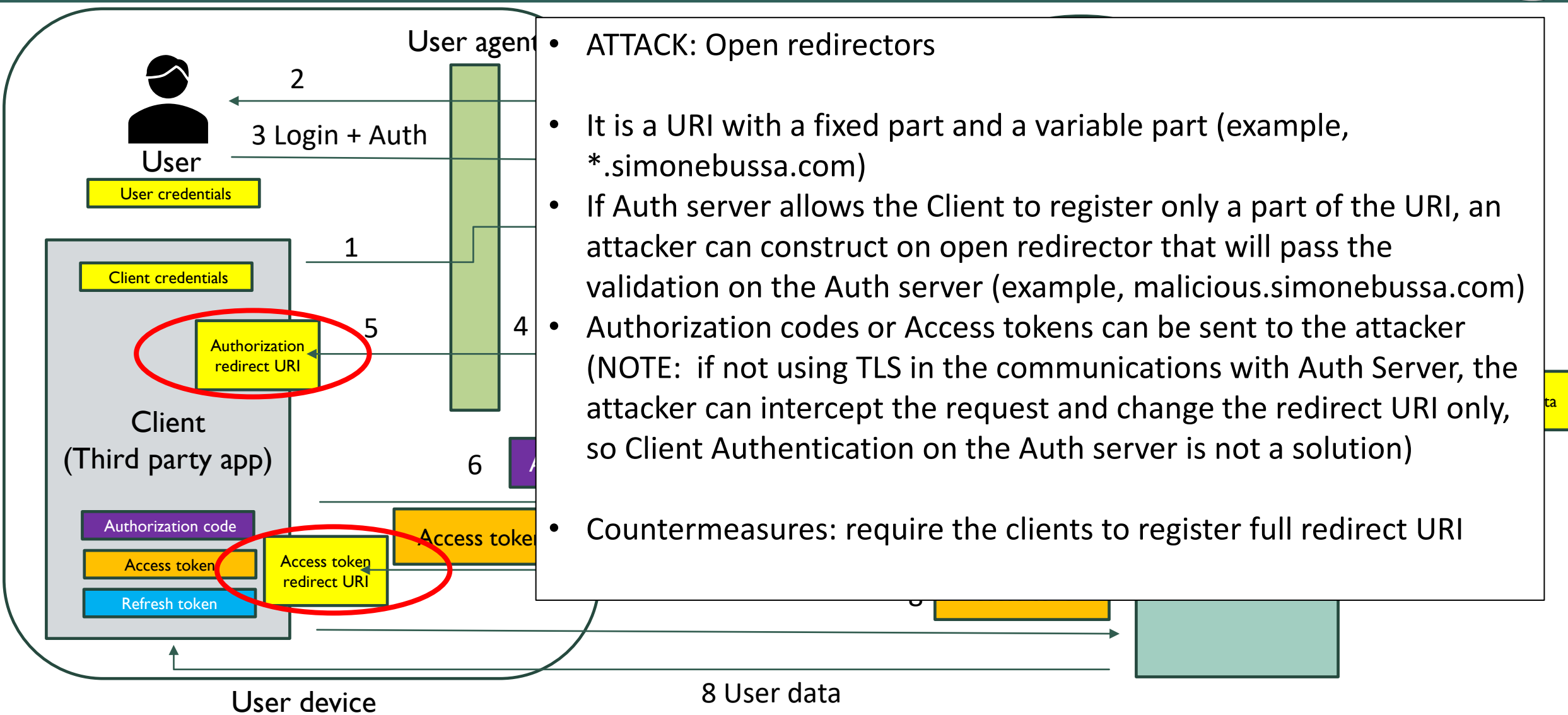




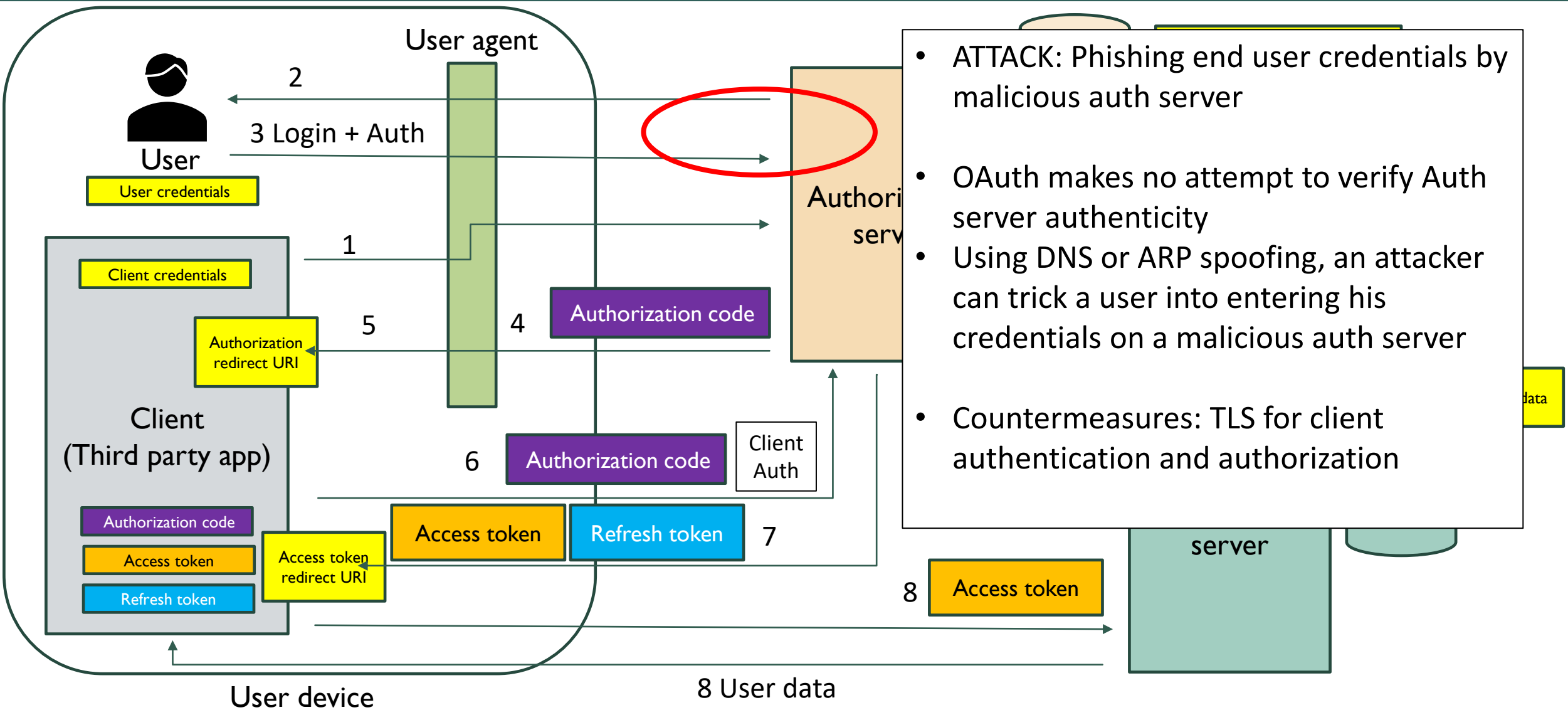
OAuth 2.0: Attacks against Client



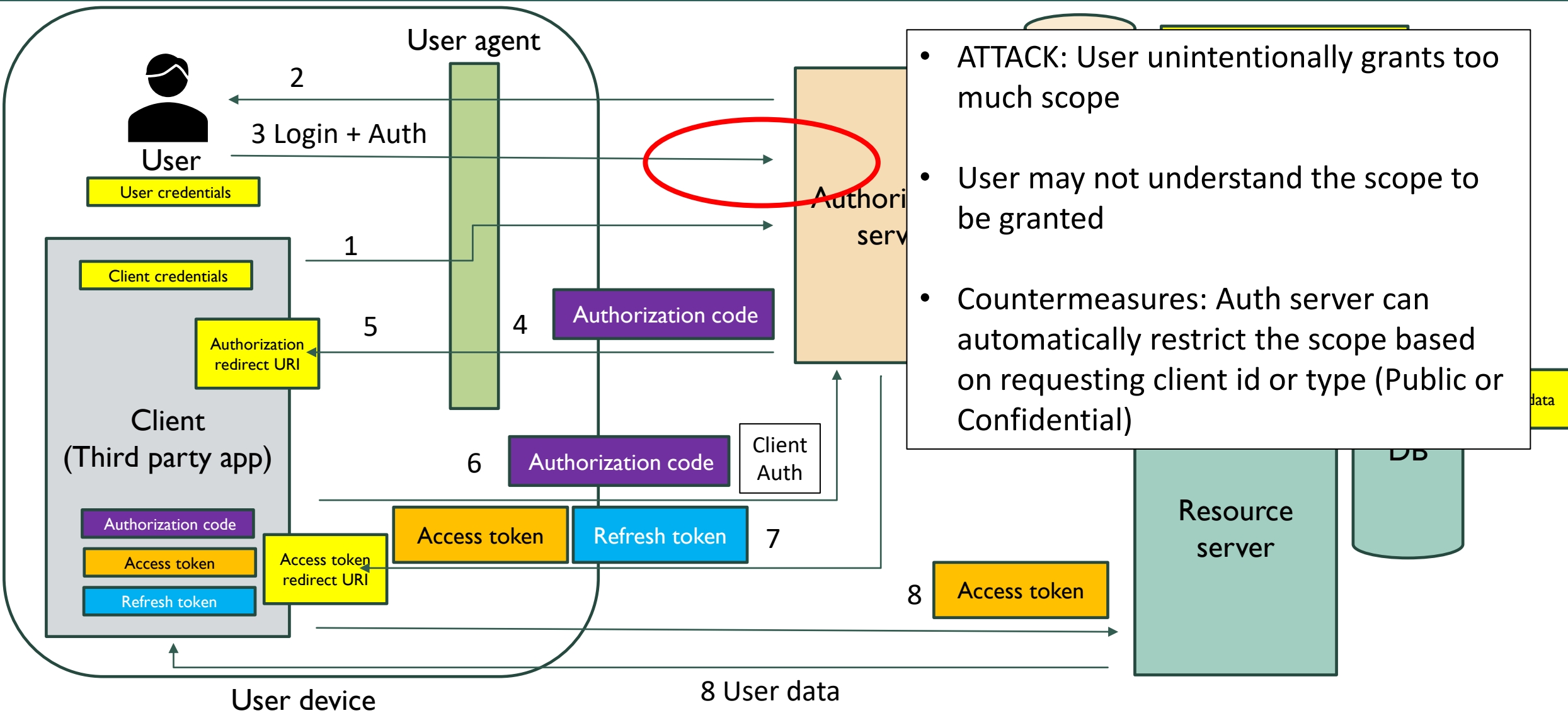
OAuth 2.0: Attacks against Client



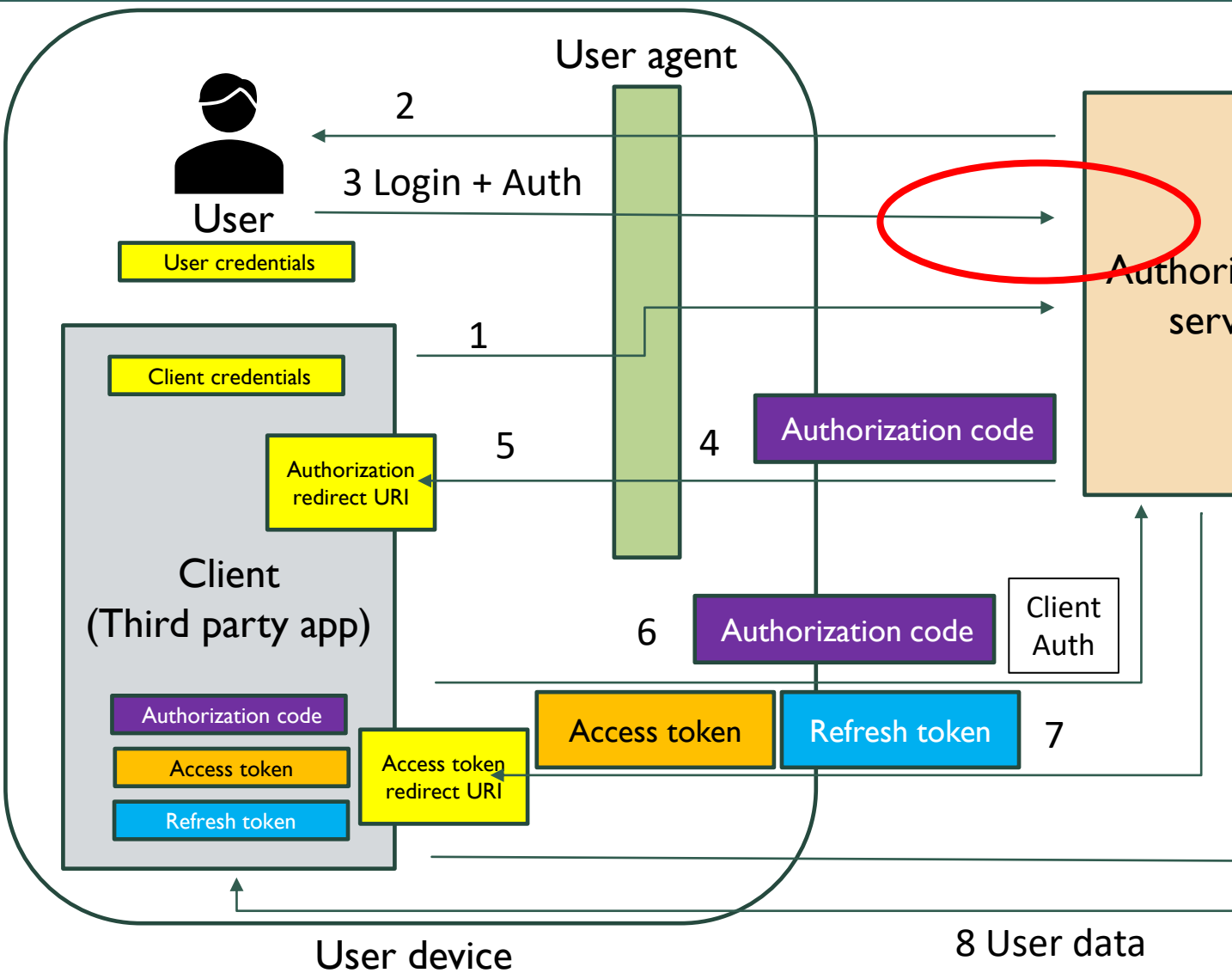
OAuth 2.0: Attacks against Auth server



OAuth 2.0: Attacks against Auth server

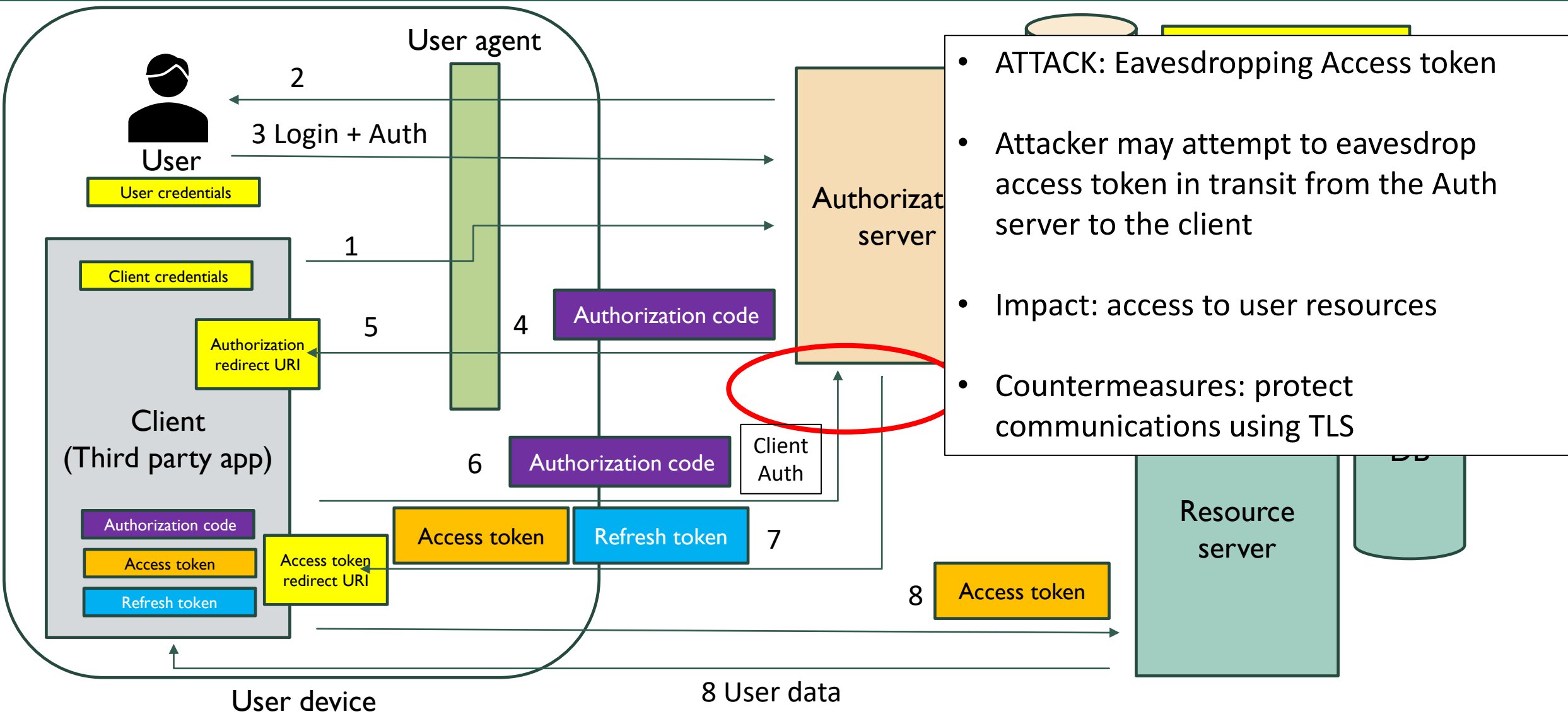


OAuth 2.0: Attacks against Auth server



- **ATTACK: Automated approvals**
- Auth server may automatically process auth requests from clients that were previously authorized by the user (without requiring login and authorization from the user again!)
- An attacker can impersonate the client and obtain the authorization code
- Countermeasures: auth server should not release automated approvals without a valid client validation (example: redirect uri validation, NOTE: Client authentication is never asked at this step, it will be required later)

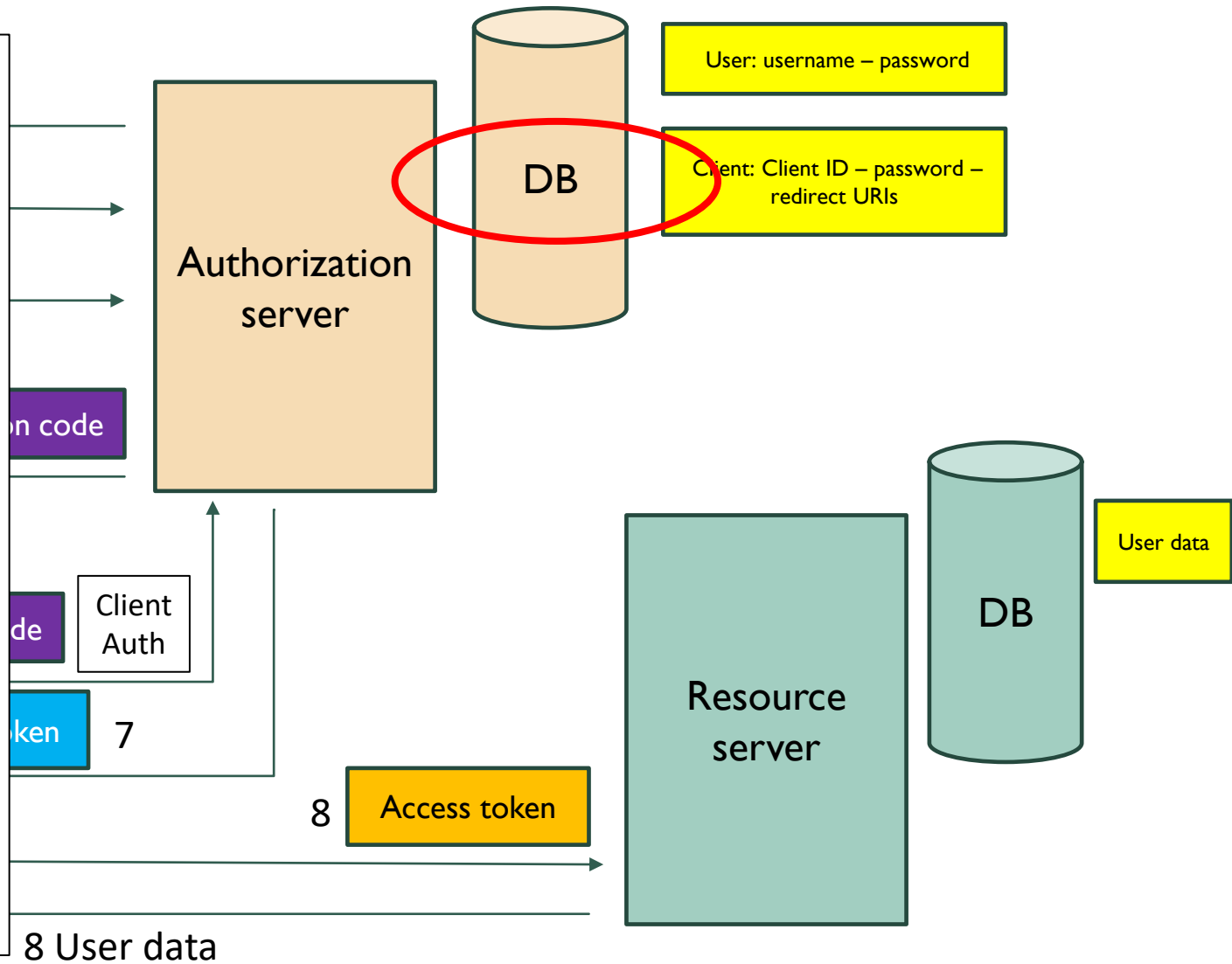
OAuth 2.0: Attacks against Auth server



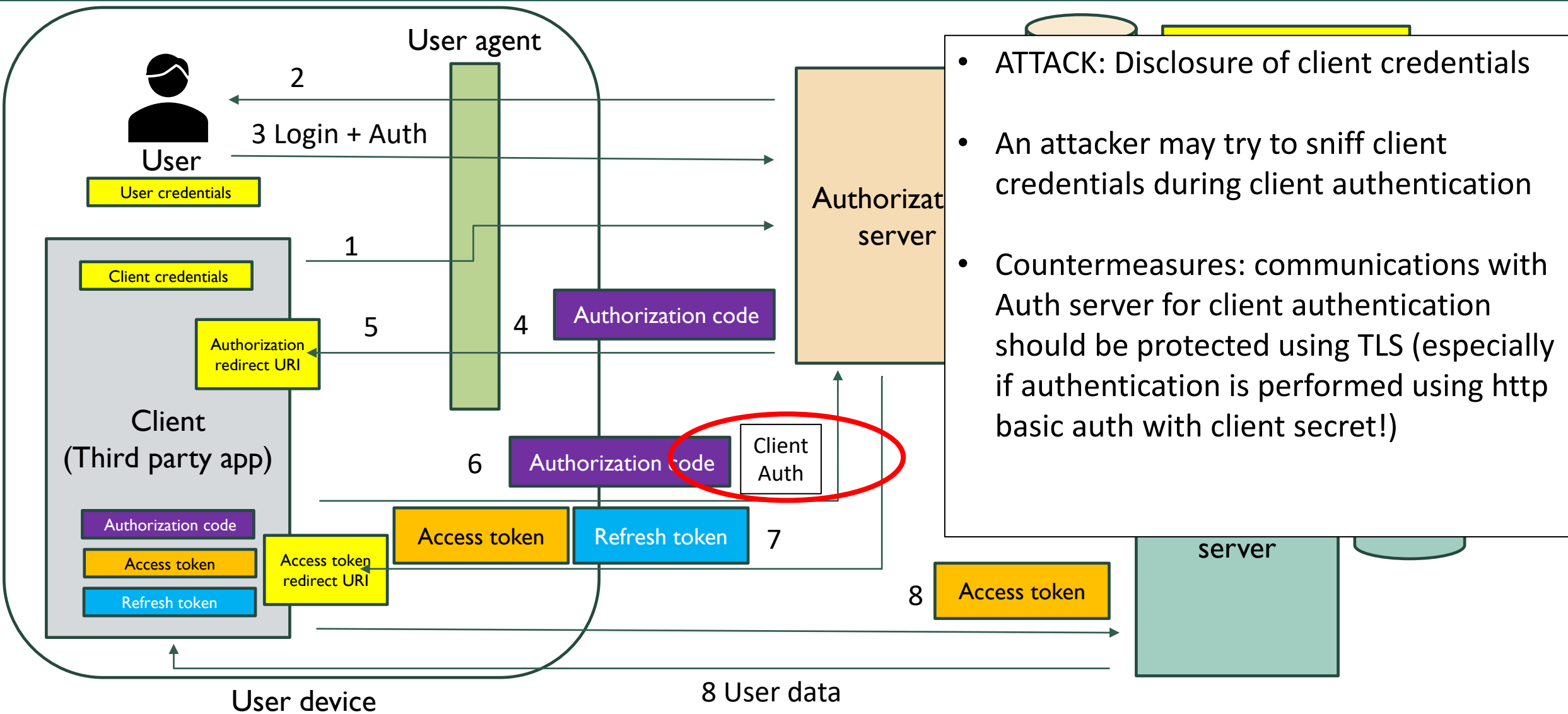
OAuth 2.0: Attacks against Auth server



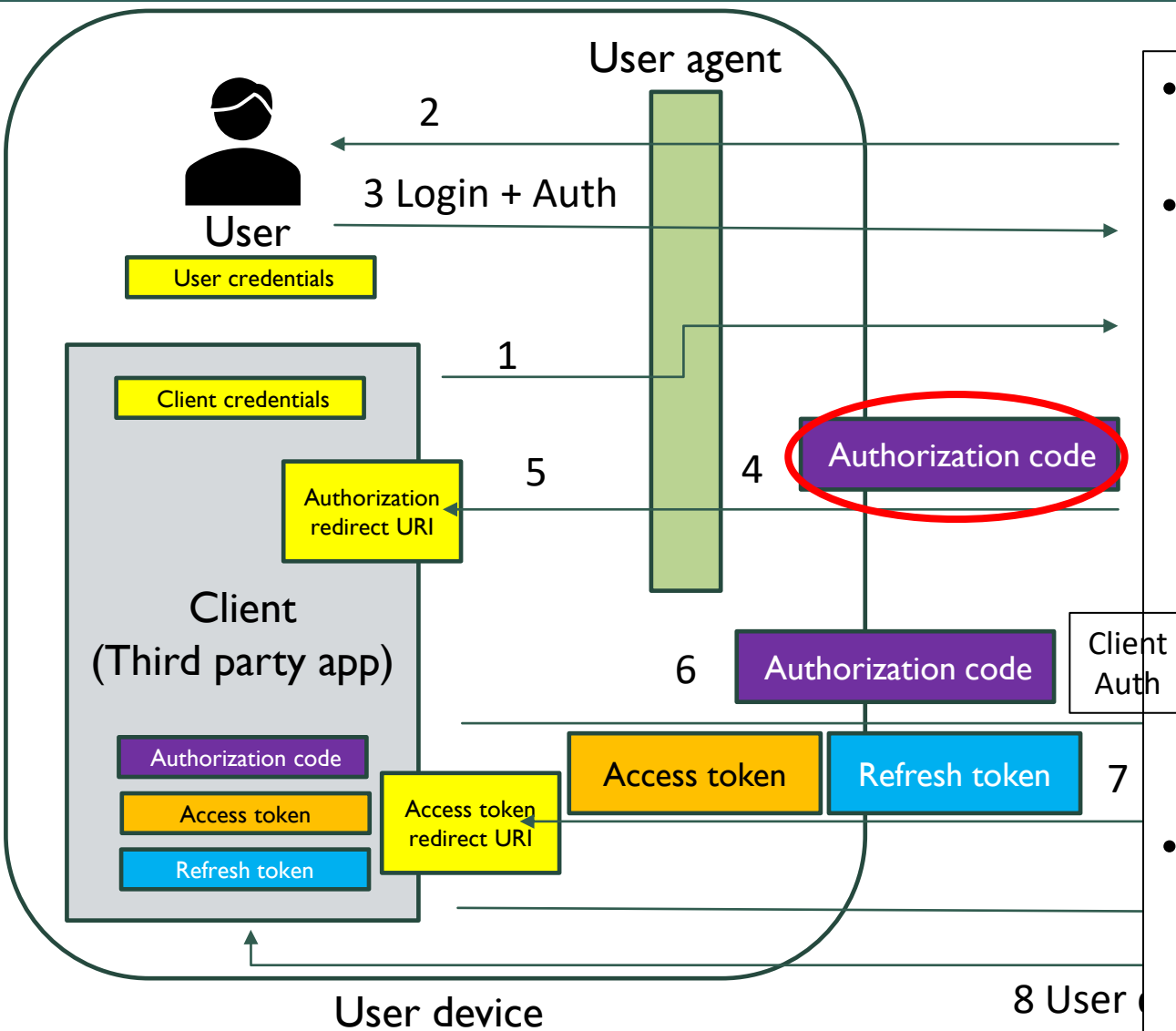
- ATTACK: Obtaining access token from Auth server database
- Two kinds of access tokens:
 - Handle (pointer to access token saved on the Auth server): easy revocation, access token non transferred to the Client, requires communication between Auth server and Resource server
 - Assertion (access token released to the client): should be protected with digital signature and encryption
- If Access token are handles, an attacker can try to stole them accessing the Auth server db (SQL injection)



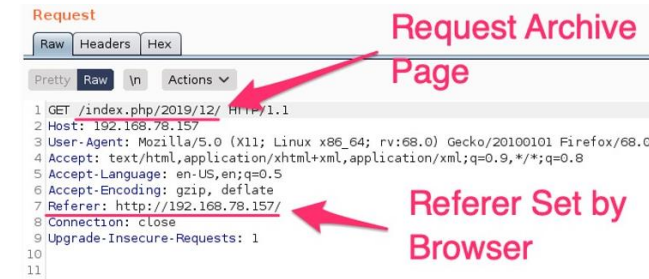
OAuth 2.0: Attacks against Auth server



OAuth 2.0: Attacks against Auth server



- **ATTACK: Eavesdropping Authorization code**
- Authorization codes are passed as query parameters in the URL of the response. As a consequence, they can be exposed to:
 - Browser history
 - Web browser request logs
 - Open redirectors (http 3XX redirect)
 - Referrer headers



- Countermeasures: TLS, binding authorization code to a specific client (and then requiring client authentication while exchanging it for access token), short-life auth code, one-time usage restriction

OAuth 2.0: Attacks against Auth server



■ Authorization code Leakage through Counterfeit client

- The attacker registers an account with a valid honest client and builds a counterfeit client (with an authorization endpoint capable of receiving Authorization codes)
- The attacker logs in the honest client and require access to an own resource on a resource server
- The client prepares the OAuth Authorization code request and sends it

Honest client Auth request for a resource on a resource server
Redirect URI: Honest client endpoint (attacker login session)

- The attacker intercepts the request, blocks it and modifies the redirect uri

Honest client Auth request for a resource on a resource server
Redirect URI: Counterfeit client endpoint

- The attacker chooses a victim among those registered on the honest client, forwards the fake request and tricks the victim into authenticating and authorizing the release of the auth code (NOTE: the victim is shown a valid request from an honest client)
- The victim authorizes the request and an auth code is returned

Auth code for a victim's resource sent to the Counterfeit client endpoint

OAuth 2.0: Attacks against Auth server



■ Authorization code Leakage through Counterfeit client

- The attacker receives the auth code on the Counterfeit client, restores the old valid Client URI and restarts the flow sending this response to the honest client that is waiting for the response to the initial attacker request

Auth code for a **victim's resource** sent to the **Honest client endpoint** within an **attacker session** on the client

- Honest client receives the response, automatically interacts with the auth server and obtains an Access token for a victim resource
- Attacker account on the honest client has then a valid access token to access a victim's resource
- Countermeasures:
 - associate the auth code with the uri of the client to which it will be delivered (inserting a field inside the auth code). In this case, the auth code is associated with the Counterfeit client uri and so it is not used by the Honest client (the Honest client understands it is a code not directed to him and so that there is an attack)
 - Auth server, before releasing the auth code, validates the redirect uri between those registered (NOTE: weak protection since the attacker may have registered the Counterfeit client as well)

OAuth 2.0: Attacks against Auth server



■ Cross site request forgery (CSRF) attack

- Also in this case, the attacker has a valid profile on the Honest client and Resource server. The attacker starts the OAuth protocol (authenticates and authorizes on the auth server) and get a valid Authorization code for an own resource on the resource server
- The attacker then stops the flow and does not deliver the auth code to the client Auth code for **attacker resource**
- The attacker tricks a victim, who had initiated the OAuth protocol with the client to get a valid resource, to follow the redirect uri of his (attacker) response and obtain the Attacker authorization code

Auth code for a **valid victim resource**

Auth code for **attacker resource**

- Victim will work on the attacker's resource: example, upload of personal data on attacker's resource, associate victim profile to attacker identity on the IdP when OAuth is used for OpenID Connect authentication (the attacker can use his own credentials to access the victim's profile)
- Countermeasures: "state" field to bind Authorization code request with the corresponding response

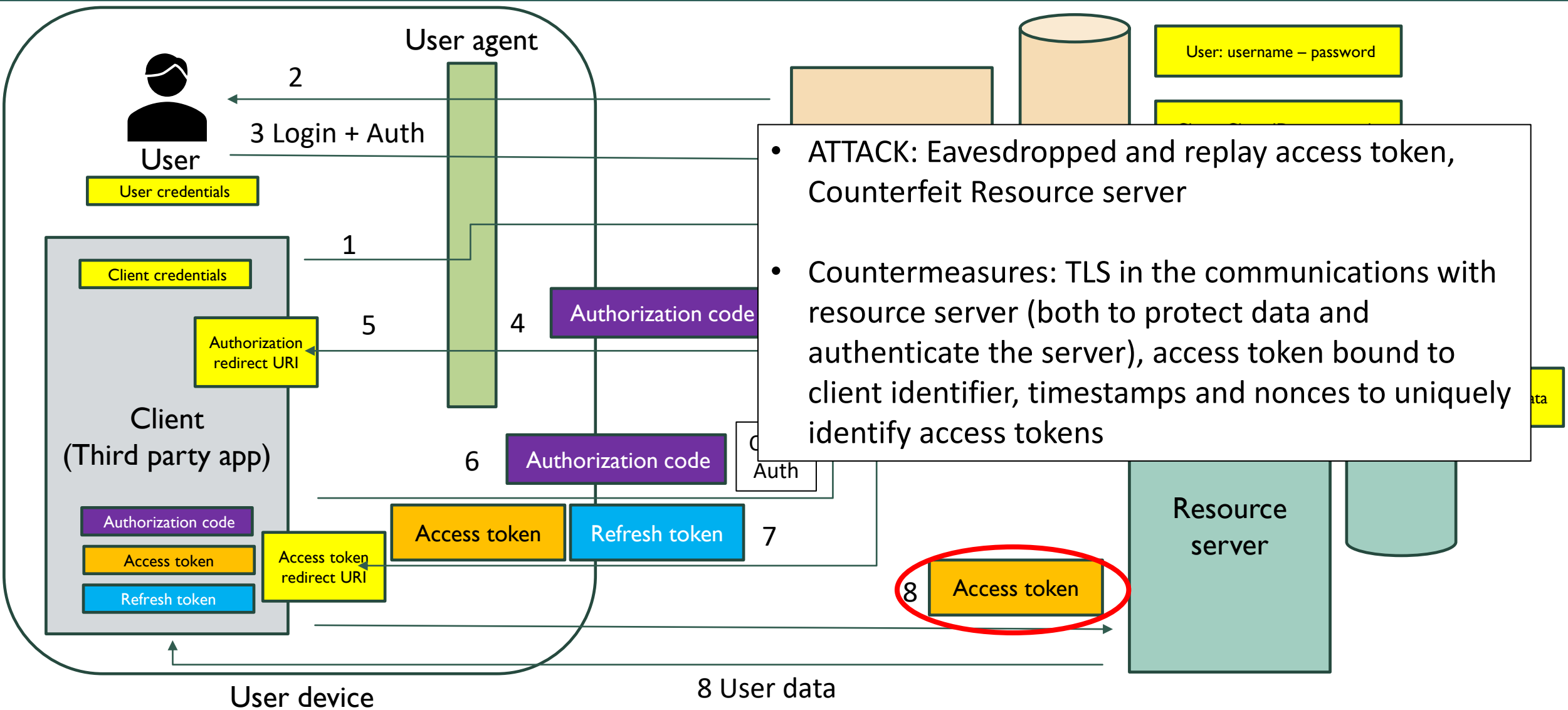
OAuth 2.0: Attacks against Auth server



■ Code substitution attack on OpenID connect

- Premise: a client can use OAuth to authenticate users (OpenID Connect profile), based on user personal info returned by the resource server. Resource server, in fact, has an endpoint from which personal info can be exchanged for a valid ID token. ID token is released together with the access token presenting a valid auth code to the Auth server (IdP)
- The attacker selects a victim client the uses an IdP to authenticate users and creates a counterfeit client that uses the same IdP
- The attackers tricks the victim to authenticate on the Counterfeit Client. The victim authenticates and it is returned a valid Authorization code to be used to retrieve victim's identity data on the resource server
- The attacker then initiates the login process on the victim client and substitutes its authorization code with that of the victim obtained on the counterfeit client
- The client automatically substitutes the auth code (of the victim) with a valid access token. The resource server accepts the access token and returns the victim identity
- The attacker is logged on the victim client in the victim profile
- Countermeasures: authorization codes should be linked to the client to which they were issued (in this example to the Counterfeit client). The Auth server, before releasing the access token, should verify that the request has been done by the same client indicated in the auth code

OAuth 2.0: Attacks against Auth server



OAuth 2.0: Client Authentication

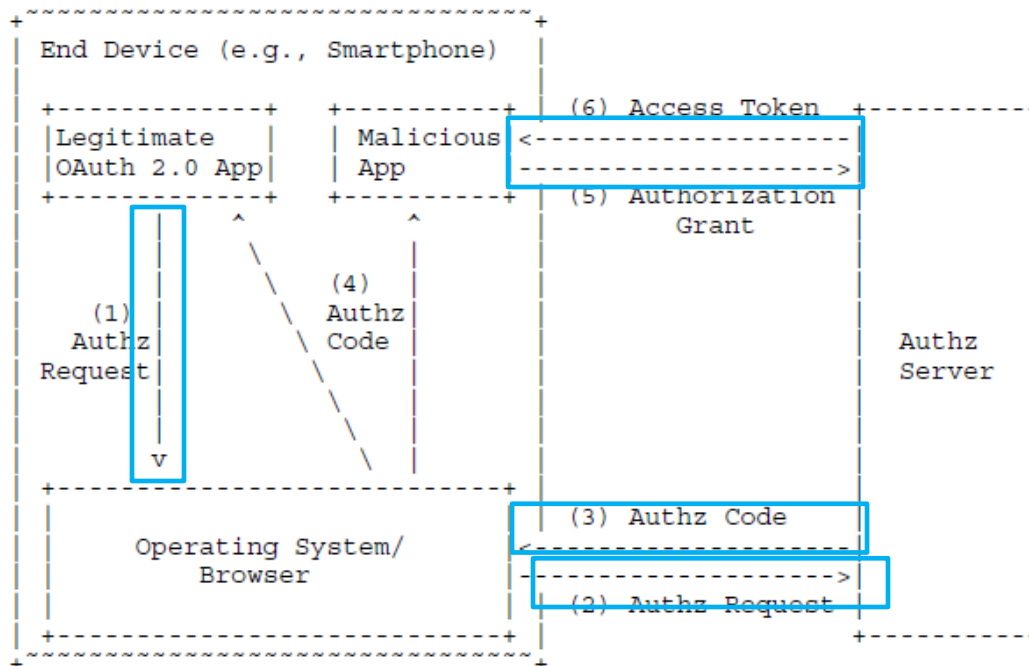


- Confidential Client authentication
 - client_secret_post: client id + client secret included as parameters in POST request (TLS required)
 - client_secret_basic: {client id:client secret} base64url encoded and inserted into the Authorization header of the request
 - client_secret_jwt: hmac of the data computed using a shared key
 - private_key_jwt: signature over data computed using an asymmetric private key
 - tls_client_auth: mutual TLS authentication of both the client and the server
 - self_signed_tls_client_auth: instead of a PKI certificate, the client uses a self-signed certificate.
NOTE: the self-signed certificate must be registered in the server in advance
- Public Client authentication
 - Cannot use any kind of secret
 - Validation of redirect URI against those registered (weak protection)

OAuth 2.0: Client Authentication



- Public client authentication using Proof Key for Code Exchange (PKCE, pixy)
 - Specification for Public clients that use Authorization Code grant without having a permanent stored secret that can be used for authentication
 - Possible attack
 1. The client native application issues an Authorization request to the browser/operating system. Done through a secure API so the request cannot be intercepted
 2. The request is forwarded by the browser operating system to the Auth server (communication protected by TLS)
 3. Auth server returns the Authorization code
 4. Authorization code is returned to the requester through the redirect URI specified in the request
ATTACK: it is possible for a malicious app to register a handler on the Operating system that responds to the redirect URI (in addition to the legitimate Client)
Both legitimate client and attacker get the Auth code



OAuth 2.0: Client Authentication

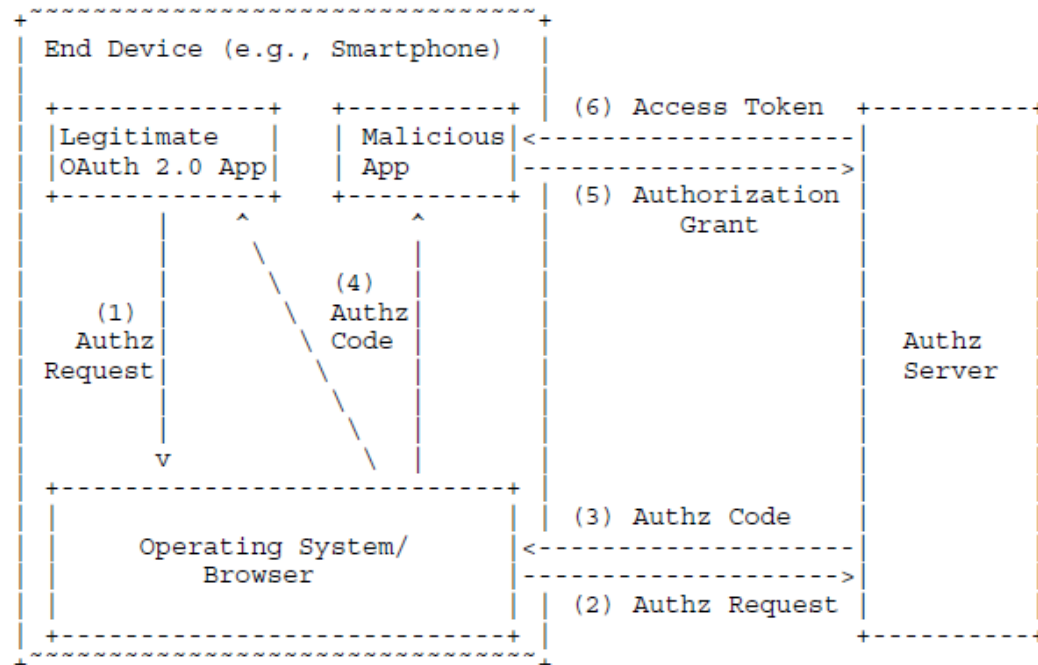


■ Public client authentication using Proof Key for Code Exchange (PKCE, pixy)

5. and 6. the malicious app can exchange the Authorization code for a valid access token (eventually authenticating with a stolen client id and client secret)

Preconditions: the attacker must register a malicious app on the same device of the victim client, the attacker must register a custom uri scheme equal to that of the victim client to handle the redirect response (and the operating

system must allow multiple applications to handle the same redirect uri), the attacker has access to client id and client secret of the legitimate client



OAuth 2.0: Client Authentication

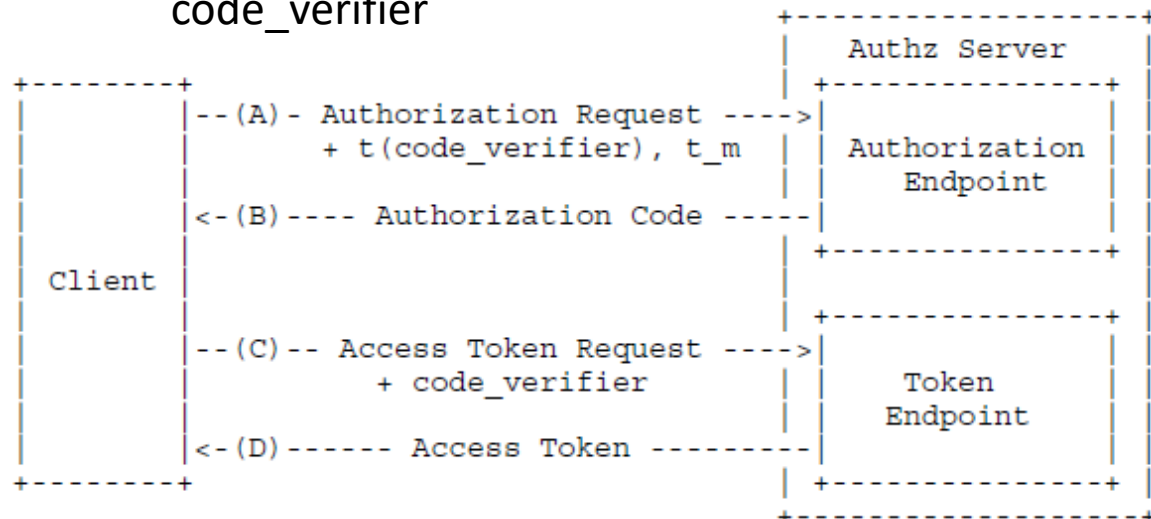


■ Public client authentication using Proof Key for Code Exchange (PKCE, pixy)

■ Solution to mitigate this attack

- `code_verifier`: dynamically created random key, unique for each Auth request
- `transformation_method (t_m)`: a transformation to apply to `code_verifier` (sha256 typically)
- `t(code_verifier)`: also called `code_challenge` that is result of applying the transformation to `code_verifier`

- (A) The client creates and record a secret `code_verifier`. It derives a transformed version `t(code_verifier)` applying the transformation method `t_m`. `t(code_verifier) + t_m` are sent to the Auth server together with Auth request
- (B) Auth server responds as usual (with the auth code) but stores the `t(code_verifier)` and `t_m`
- (C) The client, in the Access token request, send the auth code + `code_verifier` (NOTE: the attacker cannot sniff this value because it is sent over TLS)
- (D) Auth server applies `t_m` to the received `code_verifier` and check if it matches with `t(code_verifier)` received in the previous step



NOTE: an attacker that intercepts the auth code in step (B), is unable to redeem it for the access token as it does not have the `code_verifier` and cannot revert `t_m`

OAuth 2.0: Client Authentication



- Public client authentication using Proof Key for Code Exchange (PKCE, pixy)
 - Some other minor details
 - Standardized transformation: “sha256” or “plain” (code_verifier sent in clear without being transformed, NOTE: only for backward compatibility, should not be used EVEN if the Auth request is protected using TLS)
 - The auth server should maintain the association code_challenge + t_m and authorization code (so that it can be verified later). This association can be stored within the auth server db or inserted in the authorization code (encrypted and signed using auth server private keys) (stateless auth server)

Bibliography



- RFC 6749 The OAuth 2.0 Authorization framework (2012)
- RFC 6819 OAuth 2.0 Threat Model and Security Considerations (2012)
- RFC 7591 Dynamic Client Registration protocol (2015)
- RFC 7636 Proof Key for Code Exchange by OAuth Public Clients (2015)
- There are many others interesting to examine
- OAuth 2.0: Architectural design augmentation for mitigation of common security vulnerabilities (... proposes a new more complex OAuth architecture to solve all the security issues converging different implementation options into a single uniform one, to be used for all types of clients)
- The devil is in the implementation details: Empirical analysis of OAuth SSO (... describes some new vulnerabilities not present in the RFC 6819)
- FAPI part 1 (https://openid.net/specs/openid-financial-api-part-1-1_0.html)
- FAPI part 2 (https://openid.net/specs/openid-financial-api-part-2-1_0.html)
- <https://darutk.medium.com/oauth-2-0-client-authentication-4b5f929305d4> (... for OAuth Client authentication)